

Practical-6

Code:

This section contains the complete C program implementation used to solve the problem using appropriate synchronization and scheduling logic.

```
GNU nano 7.2 dining.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5 // Number of philosophers

sem_t chopstick[N];

void* philosopher(void* num) {
    int id = *(int*)num;

    while (1) {
        // Thinking
        printf("Philosopher %d is thinking\n", id);
        sleep(1);

        // Pick left chopstick
        sem_wait(&chopstick[id]);
        printf("Philosopher %d picked left chopstick\n", id);

        // Pick right chopstick
        sem_wait(&chopstick[(id + 1) % N]);
        printf("Philosopher %d picked right chopstick\n", id);

        // Eating
        printf("Philosopher %d is eating\n", id);
        sleep(2);

        // Put down chopsticks
        sem_post(&chopstick[id]);
        sem_post(&chopstick[(id + 1) % N]);
        printf("Philosopher %d put down chopsticks\n", id);
    }
}

int main() {
    pthread_t thread[N];
    int id[N];

    // Initialize semaphores
    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&thread[i], NULL, philosopher, &id[i]);
    }

    for (int i = 0; i < N; i++) {
        pthread_join(thread[i], NULL);
    }

    return 0;
}
```

```
GNU nano 7.2 dining.c
// Pick left chopstick
sem_wait(&chopstick[id]);
printf("Philosopher %d picked left chopstick\n", id);

// Pick right chopstick
sem_wait(&chopstick[(id + 1) % N]);
printf("Philosopher %d picked right chopstick\n", id);

// Eating
printf("Philosopher %d is eating\n", id);
sleep(2);

// Put down chopsticks
sem_post(&chopstick[id]);
sem_post(&chopstick[(id + 1) % N]);
printf("Philosopher %d put down chopsticks\n", id);
}

int main() {
    pthread_t thread[N];
    int id[N];

    // Initialize semaphores
    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&thread[i], NULL, philosopher, &id[i]);
    }

    for (int i = 0; i < N; i++) {
        pthread_join(thread[i], NULL);
    }

    return 0;
}
```

```
GNU nano 7.2          dining.c *
}

int main() {
    pthread_t thread[N];
    int id[N];

    // Initialize semaphores
    for (int i = 0; i < N; i++) {
        sem_init(&chopstick[i], 0, 1);
    }

    // Create philosopher threads
    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&thread[i], NULL, philosopher, &id[i]);
    }

    // Join threads
    for (int i = 0; i < N; i++) {
        pthread_join(thread[i], NULL);
    }

    return 0;
}

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo     M-A Set Mark
^X Exit      ^R Read File ^A Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo     M-G Copy
```

Result

This section illustrates how philosophers execute their actions—thinking, eating, and releasing chopsticks—while maintaining proper synchronization during program execution.

```
user@user-HP-Laptop-15-db0xxx: ~/dining_philosopher
user@user-HP-Laptop-15-db0xxx:~$ mkdir dining_philosopher
cd dining_philosopher
user@user-HP-Laptop-15-db0xxx:~/dining_philosopher$ nano dining.c
user@user-HP-Laptop-15-db0xxx:~/dining_philosopher$ gcc dining.c -o dining -lpthread
user@user-HP-Laptop-15-db0xxx:~/dining_philosopher$ ./dining
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking
Philosopher 0 picked left chopstick
Philosopher 3 picked left chopstick
Philosopher 2 picked left chopstick
Philosopher 3 picked right chopstick
Philosopher 3 is eating
Philosopher 1 picked left chopstick
Philosopher 3 put down chopsticks
Philosopher 3 is thinking
Philosopher 4 picked left chopstick
Philosopher 2 picked right chopstick
Philosopher 2 is eating
Philosopher 2 put down chopsticks
Philosopher 2 is thinking
Philosopher 3 picked left chopstick
Philosopher 1 picked right chopstick
Philosopher 1 is eating
Philosopher 1 put down chopsticks
Philosopher 1 is thinking
Philosopher 0 picked right chopstick
```